

Econ 527 HW3

Camille Bergeron

May 2026

1 Speeding up VFI

This problem takes the same form as the previous problem set. We seek to solve a simple representative agent neoclassical growth model where an agent solves

$$\begin{aligned} \max_{\{c_t\}_{t=0}^{\infty}} \mathbb{E}_0 \sum_{t=0}^{\infty} \beta^t u(c_t) \\ \text{s.t. } k_{t+1} = Ak_t^\alpha + (1 - \delta)k_t - c_t \end{aligned}$$

where $u(c) = \log(c)$, $k_t > 0$, and k_0 is given. The problem (1) can be written as the value function

$$V(k) = \max_{k' \geq 0} \log(Ak^\alpha + (1 - \delta)k - k') + \beta V(k') \quad (1)$$

We set the following parameters: $\beta = 0.96$, $A = 1$, $\delta = 1$, and $\alpha = 0.35$.

1.1 Howard Policy Iteration

To speed up Value Function Iteration, we implement the Howard Policy Iteration algorithm.

The results are shown in Table 1.

M	Time (s)	Bellman Iterations
5	9.40	34
10	7.97	23
20	7.15	15

Table 1: Results from Howard Policy Iteration

where M is defined as the number of policy iterations.

1.2 Endogenous Grid Method

For the Endogenous Grid Method, it took **2433** iterations and **1.43** seconds.

2 The Stationary Distribution of State Variables

2.1 Net Demand Using Interpolation

The transition matrix was calculated via cubic spline interpolation. Using the wealth grid from the Endogenous Grid Method, cubic spline interpolants were calculated to fit a finer wealth grid (defined as 1,000 evenly spaced grid points over the range $[a_{\min}, a_{\max}]$) onto the policy grid to approximate the policy function $g(a_i, z_j)$.

Given the policy function $g(a_i, z_j)$ and the transition matrix of z , the pair (a, z) follows a Markov process with transition matrix Υ of dimensions $ij \times ij$. The stationary distribution is then calculated using the eigenvalue method such that

$$\Upsilon^\top \phi(a_i, z_j) = \phi(a_i, z_j) \quad (2)$$

and $\sum_i \phi_i = 1$, where $\phi(a_i, z_j)$ is the stationary distribution with dimensions $ij \times 1$. The stationary distribution is the unit left eigenvector of Υ , normalized to sum to one. Uniqueness is then verified.

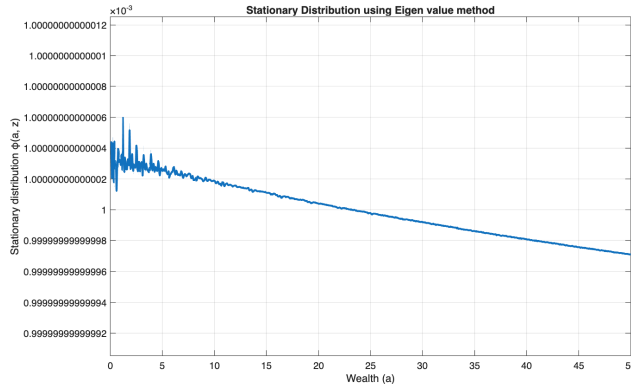


Figure 1: Stationary distribution of state variables vs. wealth

This stationary distribution is unique and took 3.46 seconds to compute. The net demand for assets is $-2.4999 \times 10^7 \approx 0$.

2.2 Net Demand Using Simulation

The net demand for assets using this method is 0.0017 and took 28.96 seconds.

3 Description of Code

Please run `PS3_run.m`. The code is formatted so that the computer does not need to re-simulate every time; instead it loads previously generated and saved

results. To see the code run from scratch (without loading saved results), delete the `results` folder found in `./programs/`. A PDF (and `.tex`) version of the output is also included (`PS3_run.pdf`).

3.1 Main

File	Description	Dependencies
<code>PS3_run.m</code>	Run file for the problem set	<code>PS3_Q1_HPI.m</code> , <code>PS3_Q1_EGM.m</code> , <code>PS3_Q2.m</code>
<code>PS3_run.pdf</code>	PDF output of <code>PS3_run.m</code>	<code>PS3_run.m</code>
<code>PS3.pdf</code>	Problem set	

3.2 Question 1: Howard Policy Iteration

File	Description	Dependencies
<code>PS3_Q1_HPI.m</code>	Sets up parameters for VFI and implements Howard Policy Iteration for $M \in \{5, 10, 20\}$	<code>rouwenhorst.m</code> , <code>polynomial_grid.m</code> , <code>howard_policy_improvement.m</code>
<code>howard_policy_improvement.m</code>	Runs Howard policy improvement algorithm to solve for value and policy functions; iteratively updates value function using fixed-policy iteration before returning to the VFI loop	<code>VFI_calc.m</code> , <code>policy_calc.m</code>
<code>VFI_calc.m</code>	Solves the Bellman equation for each (a_i, z_j) pair to obtain updated value and policy grids	<code>cubic_spline_interpolation.m</code>
<code>policy_calc.m</code>	Policy function iteration: updates the value grid V^{n+1} and policy grid a' for each (a_i, z_j) pair	<code>cubic_spline_interpolation.m</code>

3.3 Question 1: Endogenous Grid Method

File	Description	Dependencies
PS3_Q1_EGM.m	Sets up parameters and implements the Endogenous Grid Method	rouwenhorst.m, polynomial_grid.m, compute_EGM.m, calc_unconstrained.m, calc_constrained.m, cubic_spline_interpolation.m
compute_EGM.m	EGM backward step: given the exogenous grid a' and marginal utility grid $\mu(a', z) = c(a', z)^{-\gamma}$, recovers the endogenous current-period wealth grid $a(a', z)$ by inverting the Euler equation	
calc_unconstrained.m	Unconstrained EGM update: interpolates V from the endogenous wealth grid back onto the fixed exogenous grid	cubic_spline_interpolation.m
calc_constrained.m	Evaluates the Bellman equation for all a_i under the constrained policy $a' = a_{\min}$ (borrowing limit)	

3.4 Question 2

File	Description	Dependencies
PS3_Q2.m	Calculates the state transition matrix and stationary distribution; simulates the Markov chain and computes asset demand	rouwenhorst.m, cubic_spline_interpolation.m, mc_invdist.m, simulate_markov.m, simulate_a.m
simulate_markov.m	Simulates the Markov chain from the state transition matrix	mc_invdist.m
simulate_a.m	Simulates wealth and sets initial state to the borrowing limit	

3.5 General Functions

File	Description	Dependencies
rouwenhorst.m	Rouwenhorst's method for discretizing an AR(1) process	
cubic_spline_interpolation.m	Cubic spline interpolation between grid points	
mc_invdist.m	Computes the invariant distribution of a Markov chain with transition matrix Π	

4 Acknowledgments

Thanks to Professor Greaney for providing essential functions and for being very helpful throughout the course.

5 AI Usage

AI (specifically Claude) was used to debug code and format L^AT_EX.